

01. 架構與記憶體 (Memory & Architecture)

1. 值傳遞 (Pass by Value) vs 指標傳遞 (Pass by Pointer)

Go 語言中所有的參數傳遞都是值傳遞（複製）。即使是指標，傳遞的也是記憶體位址的副本。

```
flowchart LR
    subgraph Caller
        V1["user (Struct)"]
        P1["&user (Pointer: 0xC0000)"]
    end

    subgraph "Function A (Pass by Value)"
        V2["user_copy (Struct副本)"]
        V1 -- "複製整個結構體資料" --> V2
    end

    subgraph "Function B (Pass by Pointer)"
        P2["ptr_copy (Pointer副本: 0xC0000)"]
        P1 -- "複製記憶體位址" --> P2
        P2 -. "指向相同的實體" .-> V1
    end
```

2. 記憶體分配：Stack vs Heap

- **Stack (堆疊)**：函式內的區域變數，函式結束自動清除，極快。
- **Heap (堆積)**：全域變數或逃逸變數，依賴 GC (Garbage Collection) 回收，成本高。

```
block-beta
    columns 2
    block:Stack["Stack (堆疊)"]
        S1["func main() 的變數"]
        S2["func add() 的變數"]
        S3["自動釋放，無 GC 壓力"]
    end
    block:Heap["Heap (堆積)"]
        H1["被逃逸分析抓出的變數\n(例如回傳指標)"]
        H2["動態大小的 slice/map"]
    end
```

```
H3["依賴 GC 清理"]
end
```

3. 逃逸分析 (Escape Analysis)

編譯器決定變數該放 Stack 還是 Heap。當區域變數的生命週期超出了原本的函式 (例如回傳了它的指標)，它就會「逃逸」到 Heap。

```
stateDiagram-v2
    [*] --> CompilerCheck: 宣告區域變數 u := User{}
    CompilerCheck --> Stack: 是否只在函式內使用? (是)
    CompilerCheck --> Heap: 是否回傳指標 &u? (是)
    CompilerCheck --> Heap: 是否傳入 fmt.Println()? (是)

    Stack --> AutoFree: 函式 return 時自動清除
    Heap --> GC: 等待 Garbage Collector 回收
```

4. 記憶體對齊 (Memory Alignment)

結構體 (Struct) 欄位的排列順序會影響最終佔用的記憶體大小，因為 CPU 讀取記憶體時會要求特定位元組對齊。

```
packet-beta
0-7: "bool (1 byte)"
8-15: "padding (1 byte)"
16-31: "int16 (2 bytes)"
32-63: "int32 (4 bytes)"
```

(最佳化原則：將型別大小由大到小排序，可大幅減少 padding 浪費的空間)